

Relatório do Sistema AVL

Iago Cardoso Bariuka

16/10/2024

1 Descrição Geral

O sistema implementado refere-se a uma árvore AVL, um tipo de árvore binária de busca balanceada. A árvore AVL mantém a propriedade de balanceamento através da verificação e correção da altura dos nós após cada inserção ou exclusão, garantindo uma altura $O(\log n)$. A implementação trata a inclusão, exclusão e balanceamento da árvore, bem como a rotação dos nós para preservar sua estrutura balanceada.

2 Operações Implementadas

2.1 Inclusão

A função `incluir` insere um novo nó com uma determinada chave, mantendo as propriedades da árvore AVL. A implementação realiza uma busca binária para localizar o ponto correto onde a chave deve ser inserida. A busca consiste em verificar se a chave que queremos inserir é maior ou menor que a chave do nó atual, se for maior vai para o filho da direita ou se for menor vai para o filho da esquerda. Quando um nó nulo é encontrado, o novo nó com a chave desejada é criado. Depois da inserção do nó, são chamadas funções de balanceamento para verificar se é necessário restaurar o equilíbrio da árvore, utilizando rotações simples ou duplas conforme necessário e é recalculado a altura dos nós.

```
1 struct No* incluir(struct No *no, int chave);
```

2.2 Exclusão

A função `excluir` remove um nó identificado por sua chave, preservando a estrutura balanceada da árvore AVL. A implementação utiliza uma busca binária igual à função `incluir` para localizar o nó que contém a chave desejada. Após cada exclusão, a altura dos nós é recalculada e, se necessário, o balanceamento da árvore é restaurado por meio das rotações apropriadas. Existem três cenários para a exclusão:

- Nó com dois filhos: O antecessor (o maior nó da subárvore esquerda) substitui o nó a ser removido.
- Nó com apenas um filho: O filho assume a posição do nó excluído.
- Nó sem filhos (nó folha): O nó é simplesmente removido, e a função retorna NULL para que o pai se desconecte dele.

```
1 struct No* excluir(struct No *no, int chave);
```

2.3 Balanceamento e Rotações

A função `balancear` é responsável por garantir que a árvore AVL mantenha suas propriedades de equilíbrio após operações de inserção ou remoção de nós. Essa função verifica o fator de balanceamento de um nó específico e, caso necessário, aplica rotações para restaurar o equilíbrio.

```
1 struct No* balancear(struct No *raiz);
```

O fator de balanceamento de um nó é calculado como a diferença entre a altura da subárvore esquerda e a altura da subárvore direita. Se o fator de balanceamento for maior que 1 ou menor que -1, o nó está desbalanceado, e uma rotação precisa ser realizada. Existem quatro casos de desbalanceamento possíveis:

- **Rotação simples à esquerda:** Ocorre quando a subárvore direita de um nó está mais pesada. Isso é corrigido com uma rotação à esquerda.
- **Rotação simples à direita:** Ocorre quando a subárvore esquerda de um nó está mais pesada. Isso é corrigido com uma rotação à direita.
- **Rotação dupla à esquerda-direita:** Ocorre quando a subárvore esquerda de um nó está mais pesada, mas sua subárvore direita também está desbalanceada. Primeiramente, é realizada uma rotação à esquerda na subárvore esquerda, seguida de uma rotação à direita no nó atual.
- **Rotação dupla à direita-esquerda:** Ocorre quando a subárvore direita de um nó está mais pesada, mas sua subárvore esquerda também está desbalanceada. Primeiro, realiza-se uma rotação à direita na subárvore direita, seguida de uma rotação à esquerda no nó atual.

```
1 struct No* rotEsquerda(struct No *raiz, struct No* x);  
2 struct No* rotDireita(struct No *raiz, struct No* x);  
3 struct No* rotEsquerdaDireita(struct No *raiz, struct No *r);  
4 struct No* rotDireitaEsquerda(struct No *raiz, struct No *r);
```

2.4 Impressão da Árvore

A função `printree` tem como objetivo imprimir a árvore em ordem (in-order traversal), exibindo as chaves dos nós e seus respectivos níveis dentro da árvore. A função percorre a árvore de forma recursiva, primeiro visitando o filho à esquerda, depois o nó atual (raiz) e, por fim, o filho à direita. Durante o processo de visitação, para cada nó, a função imprime sua chave juntamente com o nível em que o nó se encontra.

```
1 void printree(struct No *n, int nivel);
```